

Measuring Parallel Performance: Optimizing a Concurrent Queue

How much parallel performance does each successive optimization really buy us, and why?

By Herb Sutter

Herb is a bestselling author and consultant on software development topics, and a software architect at Microsoft. He can be contacted at www.gotw.ca.

When it comes to scalability and concurrency, more is always better.

How would you write a fast, internally synchronized queue, one that callers can use without any explicit external locking or other synchronization? Let us count the ways...or four of them, at least, and compare their performance. We'll start with a baseline program and then successively apply three optimization techniques, each time stopping to measure each change's relative performance for queue items of different sizes to see how much each trick really bought us.

All versions of the code we'll consider have one thing in common: They control concurrency using (the equivalent of) two spinlocks, one for each end of the queue. Our goal in refining the code is to allow separate threads calling *Produce* and/or *Consume* to run as concurrently as possible, while also improving scalability to get more total throughput through the queue when we use larger numbers of producer and/or consumer threads.

I'll analyze the throughput of each version of the code by varying three values:

- The number of producer threads. In my test harness, each producer thread is a tight loop that calls *Produce* with no other work.
- The number of consumers. Each consumer thread is a tight loop that calls *Consume* with no other work.
- The size of the queued objects: I'll test queues of objects of two different sizes—"small" and "large" objects that each contain 10 and 100 strings, respectively, of length 26 characters, so that enqueueing or dequeueing an object costs 10 or 100 deep string copies. This actually lets us accomplish two things: a) to measure how performance depends on the size and copying cost of the queued objects; and b) to indirectly simulate the performance effect of how much "other work" the producer and consumer threads may be doing on real-world workloads, which don't usually call *Produce* and *Consume* in tight loops that just throw away the values without using them.

Let's dive in.

Example 1: Baseline

The baseline program is adapted from the code in [1] by adding a spinlock on each end of the queue to allow only one producer and one consumer to update the queue at a time. We will allow some concurrency among producer threads because the producer spinlock doesn't cover the entire body of the *Produce* function, but we will allow only one call to *Consume* to run at a time. This code follows the policy in [1] that only producer threads ever actually change the queue; consumed nodes aren't freed by the consumer, but are lazily cleaned up by the next producer.

The contained objects are held by value:

```
// Example 1: Baseline code
//
template <typename T>
struct LowLockQueue {
```

```
private:
struct Node {
    Node( T val ) : value(val), next(nullptr) { }
    T value;
    atomic<Node*> next;
};
```

The control variables include pointers to the *first* and *last* nodes in the underlying list, and a pointer to a *divider* element that marks the boundary between the producer and consumer:

```
Node *first, *last;           // for producer only
atomic<Node*> divider;         // shared
atomic<bool> producerLock;     // shared by producers
atomic<bool> consumerLock;     // shared by consumers
```

Note that, as described in [1], we always maintain at least one dummy node at the front, so the first unconsumed node is actually the one after *divider*. Any nodes before *divider* are already consumed nodes that can be lazily freed by the next producer. The constructor sets up that invariant, and the destructor (in Java or .NET, the *disposer*) simply walks the list to free it:

```
public:
LowLockQueue() {
    first = divider = last = new Node( T() );
    producerLock = consumerLock = false;
}
~LowLockQueue() {
    while( first != nullptr ) {
        Node* tmp = first;
        first = tmp->next;
        delete tmp;
    }
}
```

Consume copies the value contained in the first unconsumed node to the caller and updates *divider* to mark that the node has been consumed. Note that the entire body of *Consume* is inside the critical section, which means we get no concurrency among consumers in this version of the code:

```
bool Consume( T& result ) {
    while( consumerLock.exchange(true) )
        { } // acquire exclusivity
```

```

if( divider->next != nullptr ) {    // if queue is nonempty
    result = divider->next->value;    // copy it back to the caller
    divider = divider->next;        // publish that we took an item
    consumerLock = false;          // release exclusivity
    return true;                    // and report success
}

consumerLock = false;              // release exclusivity
return false;                      // queue was empty
}

```

Produce allocates a new node and adds *it* to the tail of the list, then performs the lazy cleanup of consumed nodes at the front of the list, if any. Note that not all of the body of *Produce* is inside the exclusive critical section—many producers can concurrently be allocating their new nodes and copying their new item's value into them, which allows some concurrency among producers:

```

bool Produce( const T& t ) {
    Node* tmp = new Node( t );    // do work off to the side

    while( producerLock.exchange(true) )
        { }    // acquire exclusivity

    last->next = tmp;              // publish the new item
    last = last->next;            // (more about this later)

    while( first != divider ) {    // lazy cleanup
        Node* tmp = first;
        first = first->next;
        delete tmp;
    }

    producerLock = false;        // release exclusivity
    return true;
}
};

```

How well would you expect this version of the code to scale for different numbers of producers and consumers, or for different kinds of queued objects? Let's find out.

Measuring Example 1

Figure 1 shows the results of running Example 1 using different numbers of producer and consumer threads, and the two sizes of queued objects. The larger the circle, the better the throughput; and each graph has a corner note showing the "max" (largest circle) and "min" (smallest circle) values on the

graph. Note the size of each circle represents the relative throughput for that graph only; in this example, even the smallest result on the left-hand graph (where the minimum value is 18,700 objects moved through the queue per second, where each object contains 10 strings) is better than the largest circle on the right-hand graph (11,300 objects per second, where each object contains 100 strings).

This test already illustrates four important properties to consider for parallel throughput. The first key property is overall throughput, or the total amount of work the system is able to accomplish. Here, we can move up to 88,000 small objects or 11,300 objects through the Example 1 queue every second.

The second key property is scalability, or the ability to use more cores to get more work done. For both sizes of queued objects, Example 1 isn't very scalable at all: We get our best throughput at two producers and one consumer, and adding more producers and consumers doesn't get more objects through the queue.

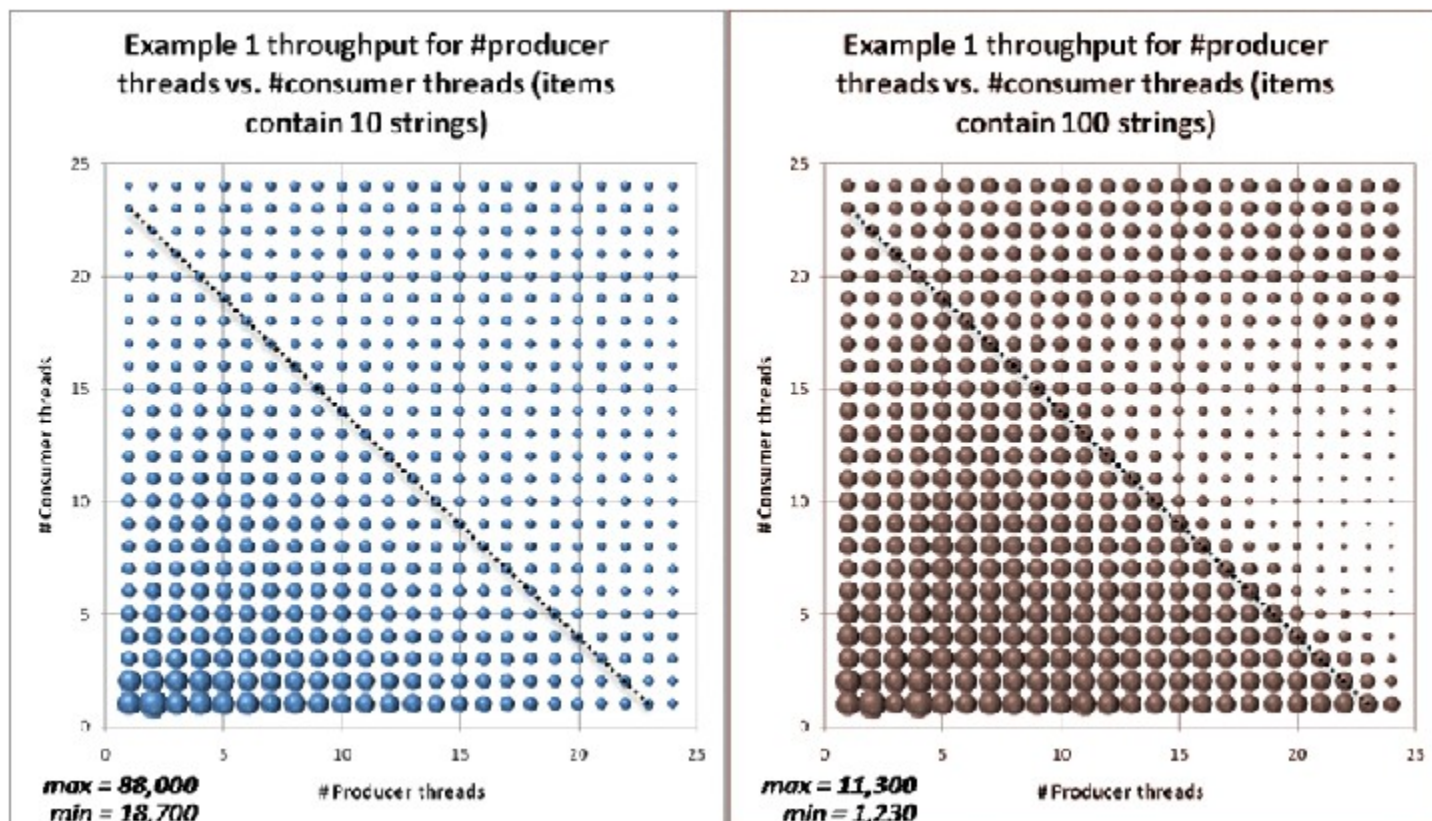


Figure 1: Example 1 throughput (total items through queue per second).

The third key property is contention, or how much different threads interfere with each other by fighting for resources. The small-object case (left graph) shows acute contention, because total throughput drops dramatically when more producers and/or consumers are added—even when those producers and consumers are running on otherwise-idle cores, which unfortunately is a great example of negative scalability. For larger queued objects, the problem is still visible but much less acute; throughput does not drop off as rapidly, at least not until we get to the dashed line.

Incidentally, speaking of that dashed line, I never described the hardware I used to run these tests. If you've been wondering about how many cores were available on my test machine, you can read the answer off the right-hand graph: The dashed line where we see a major performance cliff is where

$\#consumers + \#producers = 24$, the number of CPU cores on the test system.

The right-hand graph's diagonal dropoff is a classic artifact of the fourth key property, the cost of oversubscription, or having more CPU-bound work ready to execute than available hardware to execute it. Clearly, we can't scale to more cores than actually exist on a given machine, but there can be a real cost to exceeding that number. In this example, especially when more than half the available threads are producers, oversubscription causes a dramatic increase in churn and contention and loss of overall system throughput. In this test case, a larger number of consumers had less effect, as illustrated by the "spillover" across the top part of the graph.

Let's see if we can improve total throughput and scalability, while driving down contention and the cost of oversubscription.

Example 2: Shrinking the Consumer Critical Section

Our first optimization involves having each node allocate its contained queue item on the heap and hold it by pointer instead of by value. To experienced parallel programmers, performing an extra heap allocation might seem like a bad idea at first, because heap allocations are notorious scalability busters on many of today's memory allocators. But experience is no substitute for measurement: It turns out that, even on a system with a nonscalable allocator, the benefits often outweigh the advantages. Here, holding the queued object by pointer lets us get greater concurrency and scalability among the consumer threads, because we can take the work of actually copying its value out of the critical section of code that updates the shared data structure.

I'll show only the changed lines. In *Node* itself, of course, we need to hold the values by pointer, and adjust the constructor slightly to match:

```
// Example 2 (diffs from Example 1):  
// Moving the copying work out of  
// the consumer's critical section  
//  
struct Node {  
    Node( T* val ) : value(val), next(nullptr) { }  
    T* value;  
    atomic<Node*> next;  
};  
  
LowLockQueue() {  
    first = divider = last = new Node( nullptr );  
    producerLock = consumerLock = false;  
    //size = maxSize = 0;  
}
```

The key changes we've enabled are in *Consume*. The key is that we can move the copying of the dequeued object, and the deletion of the value, outside the critical section:

```
bool Consume( T& result ) {  
    while( consumerLock.exchange(true) )  
        {}    // acquire exclusivity
```

```

if( divider->next != nullptr ) {      // if queue is nonempty

    T* value = divider->next->value; // take it out
    divider->next->value = nullptr; // of the Node
    divider = divider->next; // publish that we took an item
    consumerLock = false; // release exclusivity

    result = *value;      // now copy it back to the caller
    delete value;
    return true;      // and report success
}

consumerLock = false;      // release exclusivity
return false;      // queue was empty
}

```

Produce only changes in its first line, where *new Node(t)* becomes *new Node(new T(t))*.

This change should allow better concurrency among consumers. But will it? Before reading on, ask yourself: How would you expect this to affect the magnitude and shape of the performance graphs? How will it affect overall throughput, scalability, contention, and the cost of oversubscription?

Measuring Example 2

Figure 2 shows the results of running Example 2 in the same test harness and on the same hardware. Again, larger circles mean better throughput on the same graph, and pay attention to each graph's "max" (largest circle) and "min" (smallest circle) notes.

Clearly, both throughput and scalability have improved across the board. Our peak throughput is better by a factor of 3 for small objects, and by an order of magnitude for large objects. From this we can deduce that Example 1's throttling of consumers was a severe limiting factor. But the reason peak throughput improved is because we improved scalability: We reach the peak now at larger numbers of producers and consumers. For large objects, we're able to saturate the 24-core machine, and get peak throughput at 15 producer threads and 9 consumer threads.

Unfortunately, for small objects we peak too early and can only usefully harness about half of the available cores before contention is the limiting factor, and adding more producers and consumers, even on otherwise-idle cores, actually accomplishes less total work. For small objects, we don't even reach the dashed line. Even for large objects, contention among consumers still seems to be a limiting factor as shown by the thinness of the top half of the graph.

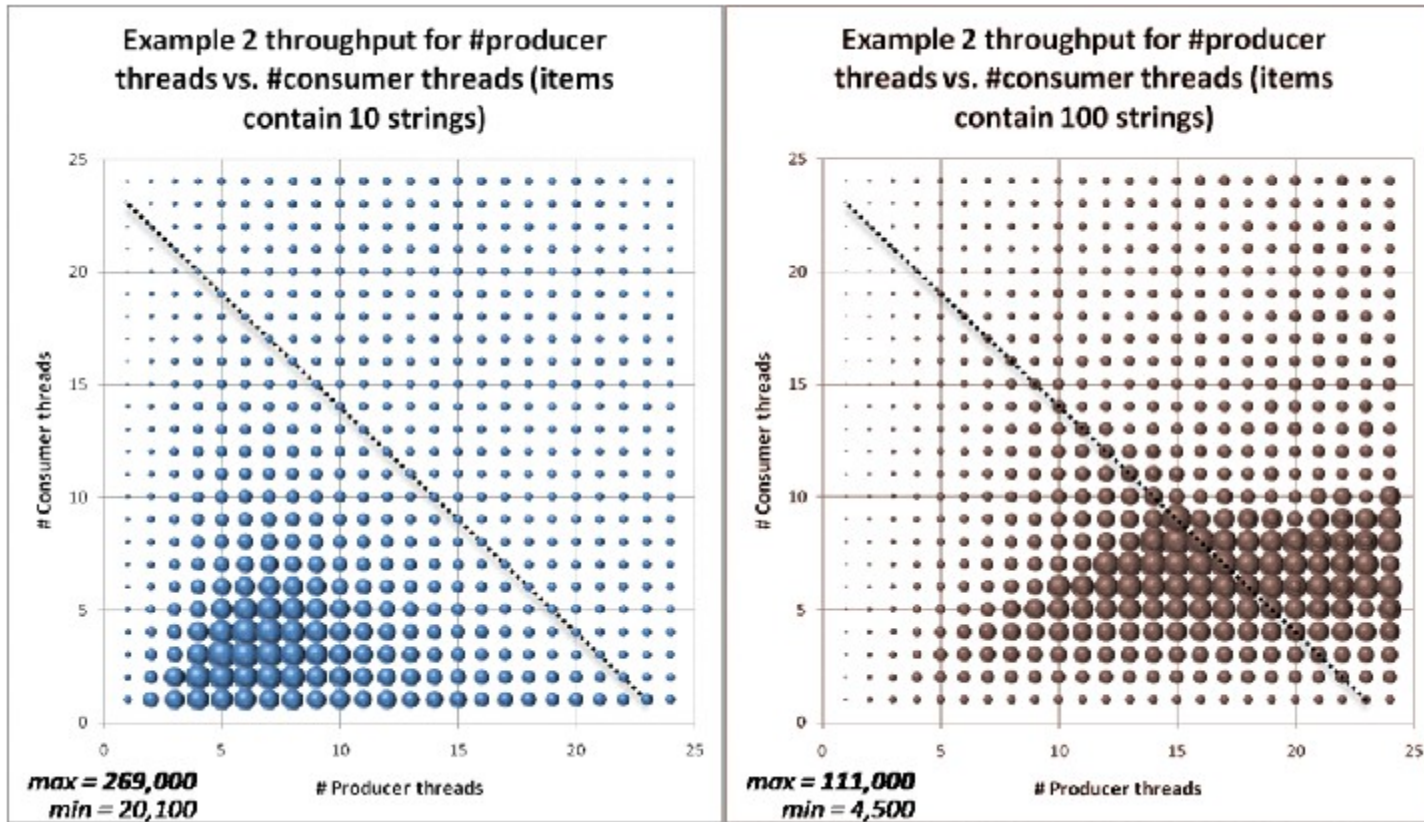


Figure 2: Example 2 throughput (total items through queue per second).

Finally, note that for large objects, not only did we reach the dashed line with throughput still rising, but we spilled across it in the lower fewer-consumers region without losing much throughput. The cost of oversubscription is lessened, at least when adding extra producers; more producers doesn't get more total work done, but they don't negatively impact total work much either. Both contention and oversubscription are still bad, however, when we add extra consumers.

That's a good first step, but let's keep going.

Example 3: Reducing Head Contention

In Examples 1 and 2, the producer was responsible for lazily removing the nodes consumed since the last call to *Produce*. But that's bad for performance for several reasons, notably because it forces a producer to touch both ends of the queue—and every thread that uses the queue, whether producer or consumer, has to touch the queue's head end. Even though a producer and a consumer don't use the same spinlocks and so can run fully concurrently with respect to each other, the fact that they touch the same memory inherently adds invisible contention, as updates to the memory containing the head nodes have to be propagated to all threads on other cores, not just to consumer threads that naturally have to touch the head end to do their work.

In Example 3, we'll let each consumer be responsible for trimming the node it consumed (which it was touching anyway) and this gives better locality. The first thing we notice is that we can get rid of *divider*—itself a source of contention because it was used by both consumers and producers:

```
// Example 3 (diffs from Example 2):
// Moving cleanup to the consumer
//
LowLockQueue() {
    first = last = new Node( nullptr ); // no more divider
    producerLock = consumerLock = false;
}
```

Consume now doesn't need to deal with *divider*, but must add the work to clean up the previous now-unneeded *first* dummy node when it consumes an item:

```
bool Consume( T& result ) {
    while( consumerLock.exchange(true) )
        { } // acquire exclusivity

    if( first->next != nullptr ) { // if queue is nonempty
        Node* oldFirst = first;
        first = first->next;
        T* value = first->value; // take it out
        first->value = nullptr; // of the Node
        consumerLock = false; // release exclusivity

        result = *value; // now copy it back
        delete value; // and clean up
        delete oldFirst; // both allocations
        return true; // and report success
    }

    consumerLock = false; // release exclusivity
    return false; // queue was empty
}
```

Next, *Produce* becomes simpler because we can eliminate the lazy cleanup code. However, just eliminating that code leads to a very subtle pitfall because one existing line also has to change. Can you see why?

```
bool Produce( const T& t ) {
    Node* tmp = new Node( t ); // do work off to the side
```



```

while( producerLock.exchange(true) )
{ }           // acquire exclusivity

last->next = tmp;           // A: publish the new item
last = tmp;                // B: not "last->next"

producerLock = false;      // release exclusivity
return true;
}

```

Changing Responsibilities Can Introduce Bugs

Note that line *B* used to be `last = last->next;`. That was always slightly inefficient because it needlessly reread `last` (a holdover from the original code written by someone else). Now, if left unchanged, it becomes something much worse: a small race window. Now that there's no *divider* and consumers clean up consumed nodes, the way consumers know there's an item available to be consumed is to check `first->next`; if it's not null, it's okay to go ahead and consume a node—and delete what used to be the first one because that node is no longer needed. The trouble arises when a sequence like the following occurs:

- Initially: queue is empty, `first == last`
- The producer (from Example 2 code, without the Example 3 correction): `last->next = tmp; // A: publish`
- The consumer performs an entire call to *Consume* the just-published node, including deleting the now unnecessary previous first node before it
- Then the producer dereferences `last`
`last = last->next; // B: update last`
// oops: accesses freed memory.

The key is that the act of publishing the new node (line *A*) not only advertises that the new node is ready to be consumed, but also implicitly transfers ownership of the preceding node to the consumer. Hence, line *B* must not dereference `last` again, but should just assign from `tmp` directly.

"But," someone might object, "will this interleaving really happen? After all, *A-B* is a very small window for a call to *Consume* to fit into." True, it won't happen often. Based on experience, however, I can report that under heavy stress on a multicore system, this tends to fail once for every few tens of millions of items moving through the queue. This was the only race I wrote (that I know of) when putting these examples together, and it was a real pain to reproduce and diagnose.

Moral: When you change responsibilities for cleanup, code that used to be innocuous can suddenly turn into a subtle race window.

Measuring Example 3

But back to the main event: How well does moving the cleanup responsibility and reducing contention on the head of the queue really help? Again, before seeing my results, consider how much, and why, you think this is likely to affect throughput, scalability, contention, and the oversubscription penalty.

Figure 3 shows the Example 3 performance results. The effects are mainly on the left-hand small object graph, with only incremental improvements for large objects. For small objects, peak throughput has improved by nearly another factor of two, and we've again improved scalability and actually get close to reaching the dashed line, which represents our capacity for getting more work done using more cores. There is some dropoff due to contention as we exceed about 20 active threads (e.g., 12 producers and 8 consumers), and for the first time we can actually see the oversubscription wall on the left-ha

graph beyond 24 threads. Although we'd like to scale that wall, right now we're happy to just be able to approach it in the first place!

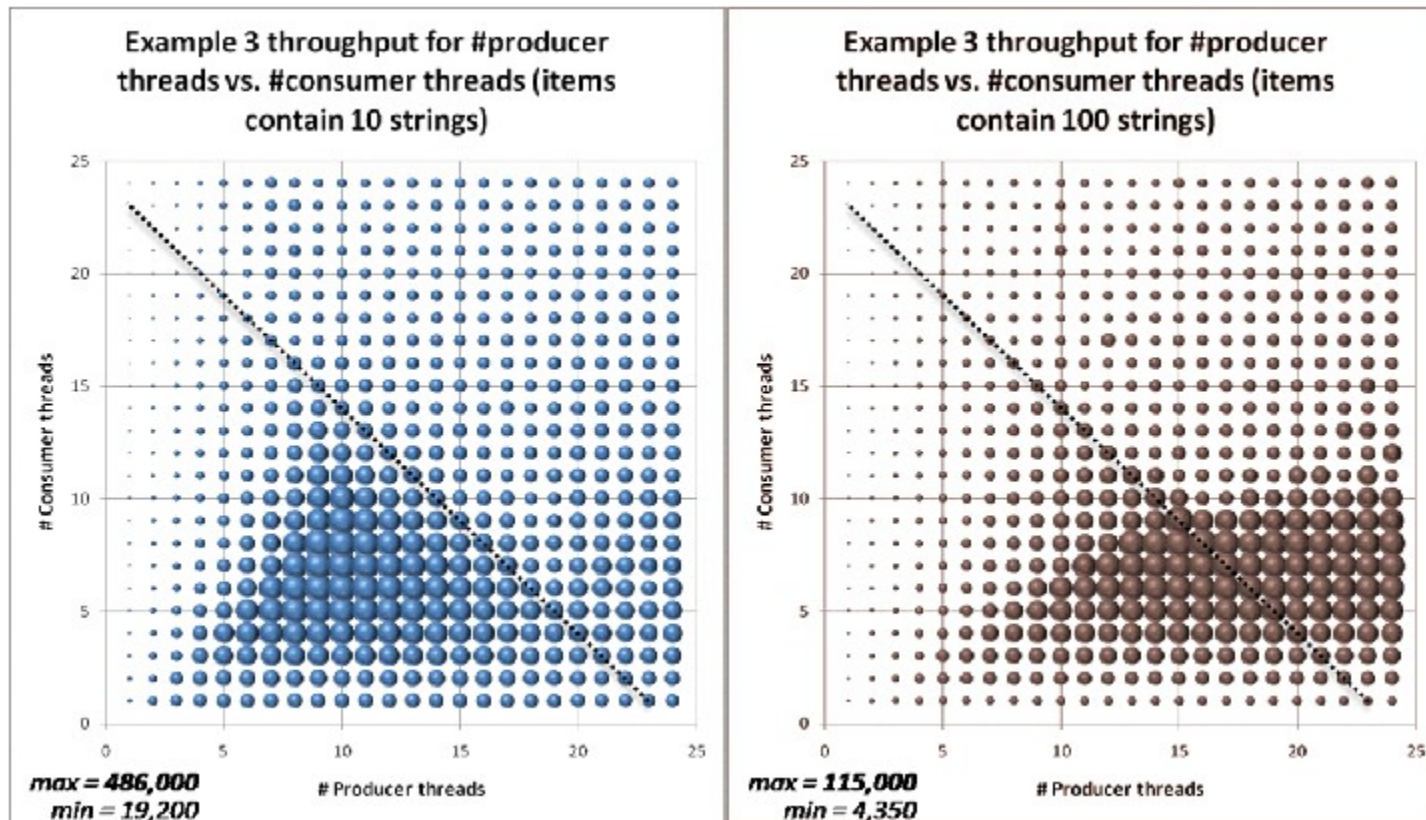


Figure 3: Example 3 throughput (total items through queue per second).

Example 4: Padding To Avoid False Sharing

The changes for our final Example 4 are extremely simple compared to the changes in Examples 2 and 3. In fact, we're not going to make any changes to the program logic at all: We're simply going to follow the advice that "if variables *A* and *B* are...liable to be used by two different threads, keep them on separate cache lines" to avoid false sharing or "ping-ponging," which limits scalability [2]. In this case, we want to add padding to ensure that different nodes (notably the *first* and *last* nodes), the *first* and *last* pointers to those nodes, and the *producerLock* and *consumerLock* variables are all on different cache lines:

```
struct Node {
    Node( T* val ) :    value(val), next(nullptr) { }
    T* value;
    atomic<Node*> next;
    char pad[CACHE_LINE_SIZE - sizeof(T*) - sizeof(atomic<Node*>)];
};
```

```
char pad0[CACHE_LINE_SIZE];
Node* first;           // for one consumer at a time
char pad1[CACHE_LINE_SIZE - sizeof(Node*)];
atomic<bool> consumerLock; // shared among consumers
char pad2[CACHE_LINE_SIZE - sizeof(atomic<bool>)];
Node* last;           // for one producer at a time
char pad3[CACHE_LINE_SIZE - sizeof(Node*)];
atomic<bool> producerLock // shared among producers
char pad4[CACHE_LINE_SIZE - sizeof(atomic<bool>)];
```

That seems like a pretty trivial change. Is it really worth its own section? Again, ask yourself: Is this likely to have any effect on throughput, scalability, contention, or oversubscription? If so, how much, and why?

Measuring Example 4

Figure 4 shows the Example 4 performance results. Again, the effects are less noticeable for large objects, where the cost of copying dominates and was already moved out of the critical section in earlier code revisions.

For small objects, peak throughput has improved by another 20 percent over Example 3 by extending scalability right to the dashed line representing the total number of hardware cores. As a bonus, we've also reduced the contention dropoff to the dashed line and beyond, smoothing the transition into oversubscription. As in every case, the sparse top half of each graph shows that larger numbers of consumers are still causing significant contention, but we've pushed the consumer contention effect out (upwards) somewhat for both small and large objects.

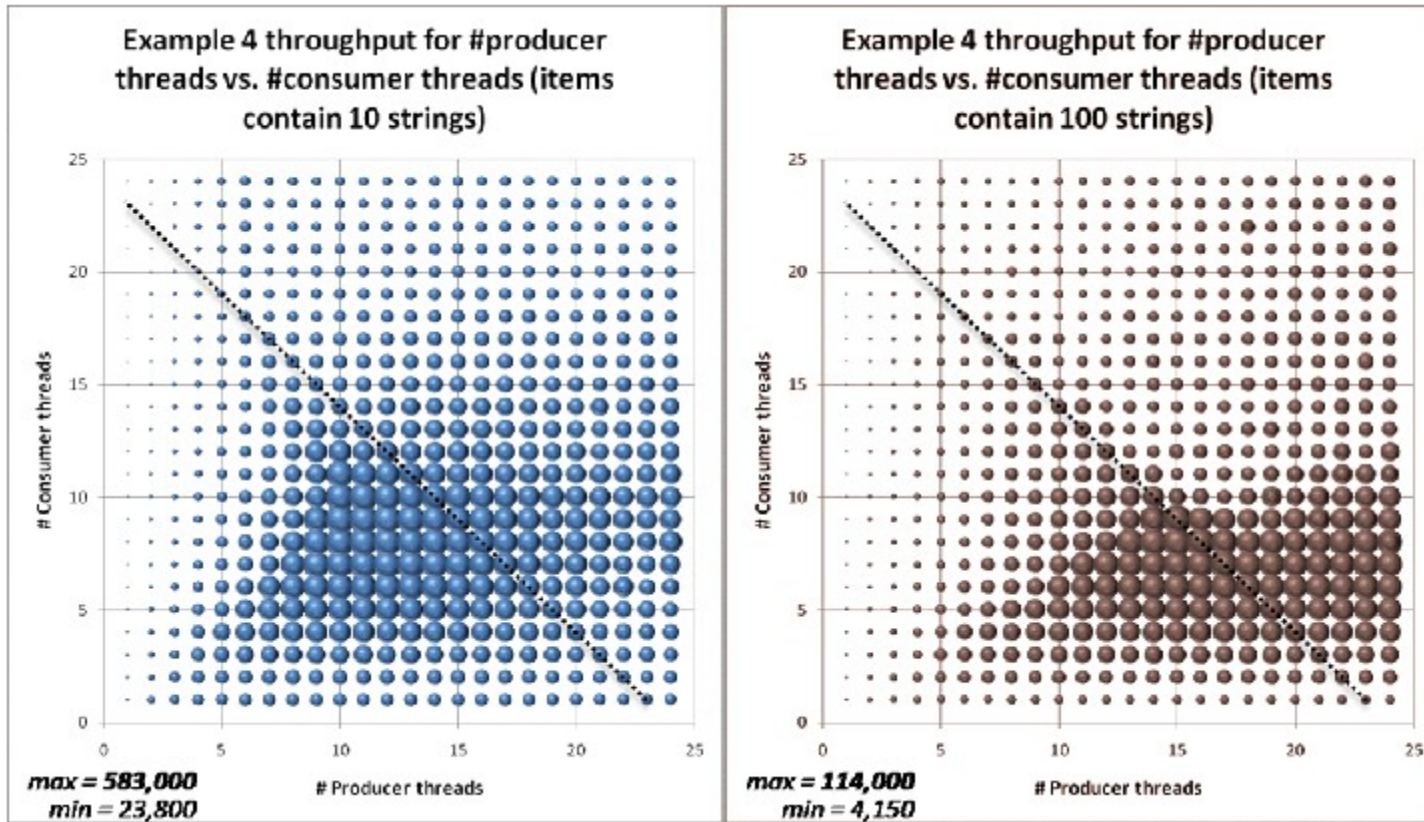


Figure 4: Example 4 throughput (total items through queue per second).

Direct Comparisons

Graphs like Figures 1-4 are useful to get a sense of the shape of our code's scalability for varying amounts of different kinds of work, and allow some visual comparison among different versions of the code. But let's consider a more direct comparison by graphing Examples 1-4 together on a line graph. Because all of the graphs so far show that our code tends to favor having more producers than consumers, I've chosen to analyze the slice of data along the line $\#producers = 2 * \#consumers$ (i.e., $2/3$ of threads are producers, $1/3$ are consumers), which goes through the fat high-throughput part of each of the previous graphs to allow for fairer comparison.

Figure 5 shows each example's throughput along this selected slice for small queue items, with automatically generated polynomial trend lines to connect the dots. Notice that each vertical slice contains several dots of the same example; this is because I ran each test several times, and the vertical spread in dots shows performance variability in repeated executions of the same test.

First, consider the left-hand graph in Figure 5, which shows the straightforward throughput numbers: As we now know to expect, Example 1 clearly has the worst performance, and each of the other examples successively improve both the peak throughput (the magnitude of each curve's high point) and scalability (how far to the right each peak is). You can visually see how each successive change we made removed an invisible barrier and let the curve continue growing a bit further. The downslope on the right-hand side of each curve shows the throughput-slowing effects of contention. Oversubscription would show up on this graph as a discontinuity as we pass 24 threads; here, with small objects, it's not very visible except for Example 3, as we also see

earlier.

linear scaling
(approx)

cores on
test machine

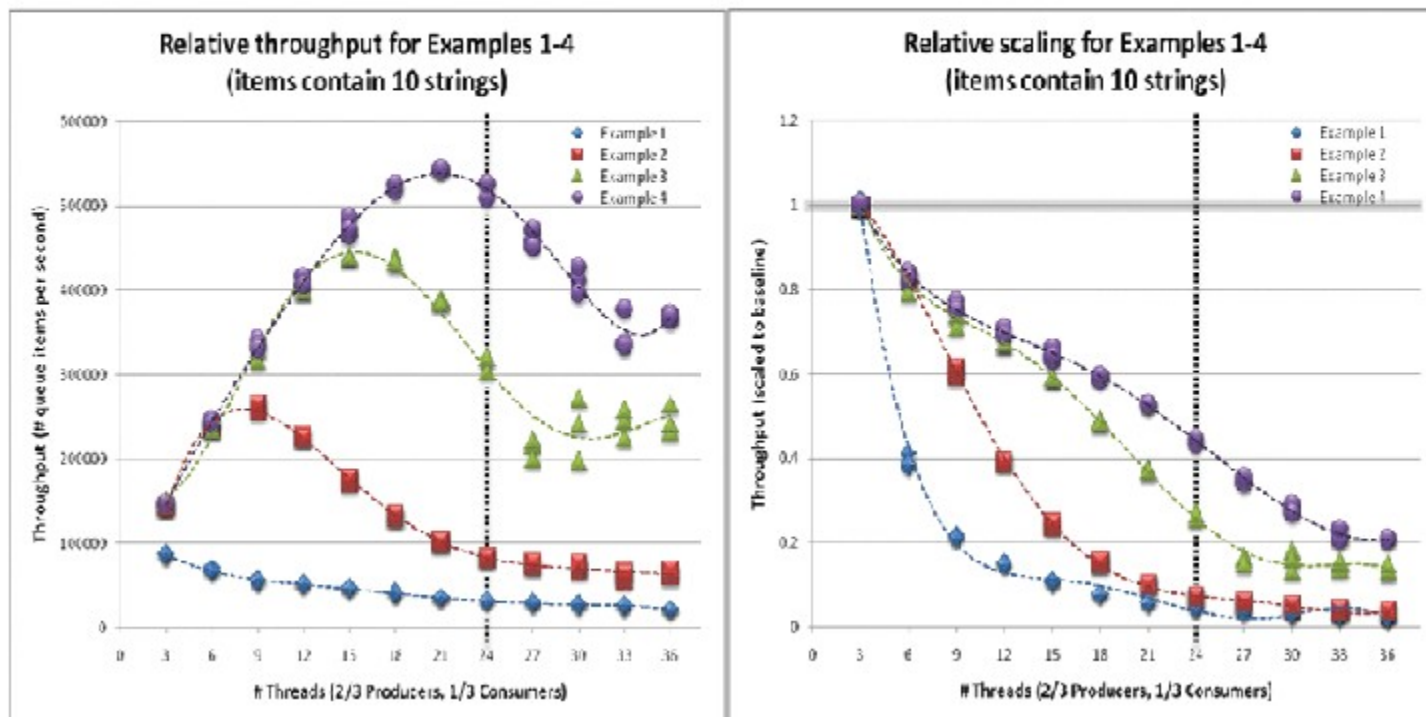


Figure 5: Comparing throughput and scaling for Examples 1-4 along the line #Producers = 2 * #Consumers (small queue item size).

But there's another useful way to scale the graph. Still in Figure 5, the right-hand graph shows exactly the same data as the left-hand graph, only we scale it by dividing each line's data by that same line's initial value and the number of threads. This is a more direct way to measure the relative scalability of each version of the code—the ideal is the grey horizontal bar, which shows linear scalability, and we want our curve to float as high and as close to that as possible. As before, Example 1 is the worst, because it quickly dives to awful depths. Example 4 is the best: It's still delivering over 50 percent scalability until just before the machine is fully saturated—getting nearly 4 times the performance using 7 times the cores (21 versus the baseline 3) may not be linear scaling, but it's still fairly decent scaling and nothing to sneeze at.

Now consider Figure 6, which shows each example's throughput along the same slice, but this time for large queue items. Again, the dotted curves are automatically generated polynomial trendlines to connect the dots; we'll come back to these in a moment.

*linear scaling
(approx)*

*# cores on
test machine*

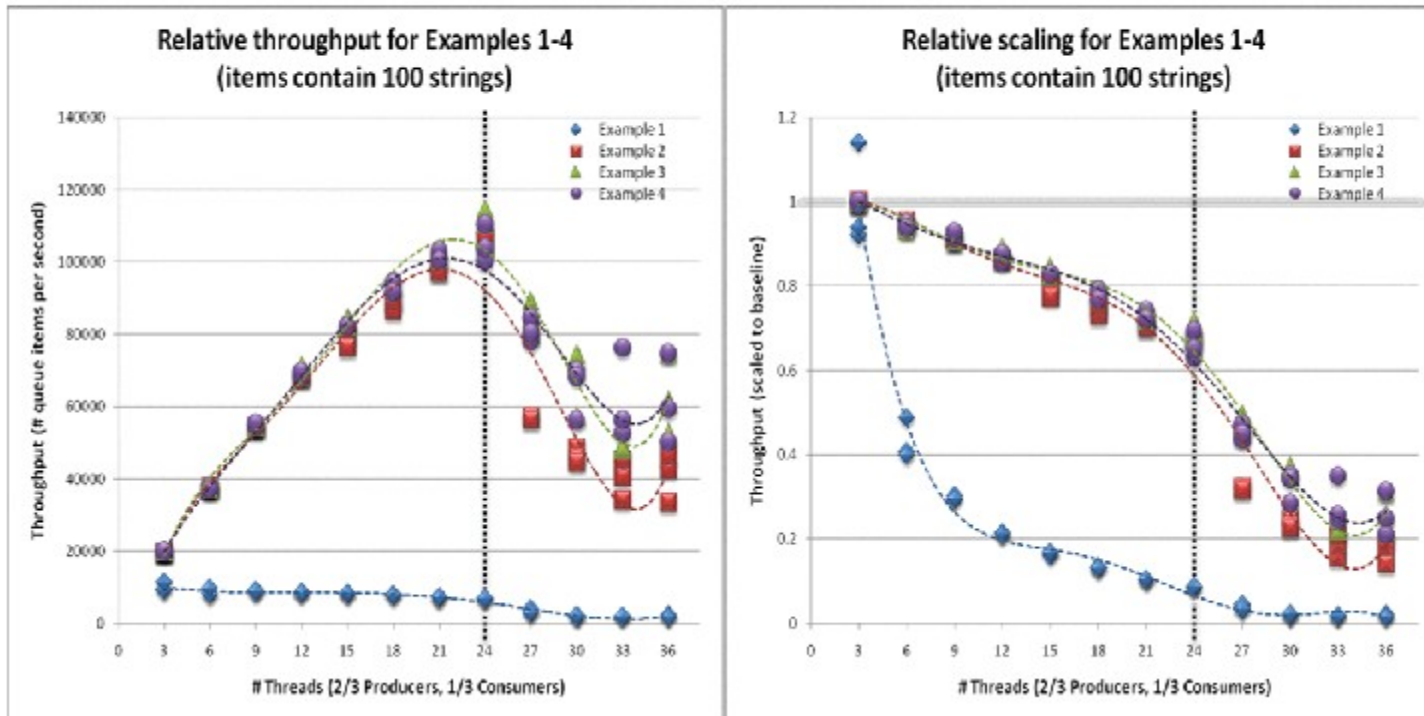


Figure 6: Comparing throughput and scaling for Examples 1-4 along the line #Producers = 2 * #Consumers (large queue item size).

What can we conclude from Figure 6? There's much less difference among Examples 2 through 4 for larger queued items where the cost of copying dominates—or, equivalently, when the producer and consumer threads are doing more work than just madly inserting and removing queue items as quickly as possible and throwing them away. Most of the benefit came from just applying the change in Example 2 to hold the queued item by pointer in order to gain more concurrency in the consumer, thereby removing what evidently was a consumer bottleneck. On the right-hand graph it's evident that all examples scale about equally well for large objects, and all are much better than they were in Figure 5 for small objects—70 percent scaling until right before we achieve machine saturation. And those pretty polynomial trendlines help us visualize what's going on better...

Or do they?

A Word About Trendlines

Beware automatic trendlines: They are often useful, but they can also lie.

Look closely again at Figure 6: The trendlines actually obscure what's really going on by creating a false visual smoothness and continuity that our human eyes are all too willing to believe. Try to imagine the graph without any trendlines, and ask yourself: Are these really the trendlines you would draw? Ye they're close to the data up to about 21 threads. But they under-represent throughput and scalability at the machine-saturation point of 24 threads, an

then don't account for the sudden drop and increased variability beyond saturation.

One of the weaknesses of curve-fitting trendlines is that they expect all of the data to fit one consistent pattern, but that is often only true within certain regions. Trendlines don't deal well with discontinuities, where we shift from one region to another one with fundamentally different characteristics and assumptions. Clearly, trying to use fewer cores or more cores than exist on the actual hardware are two different regions, and what the data actually correctly shows is that there's a jump between these regions).

If we replace the automatically generated trendlines with hand-fitted ones, a very different and much truer picture emerges, as shown in Figure 7.

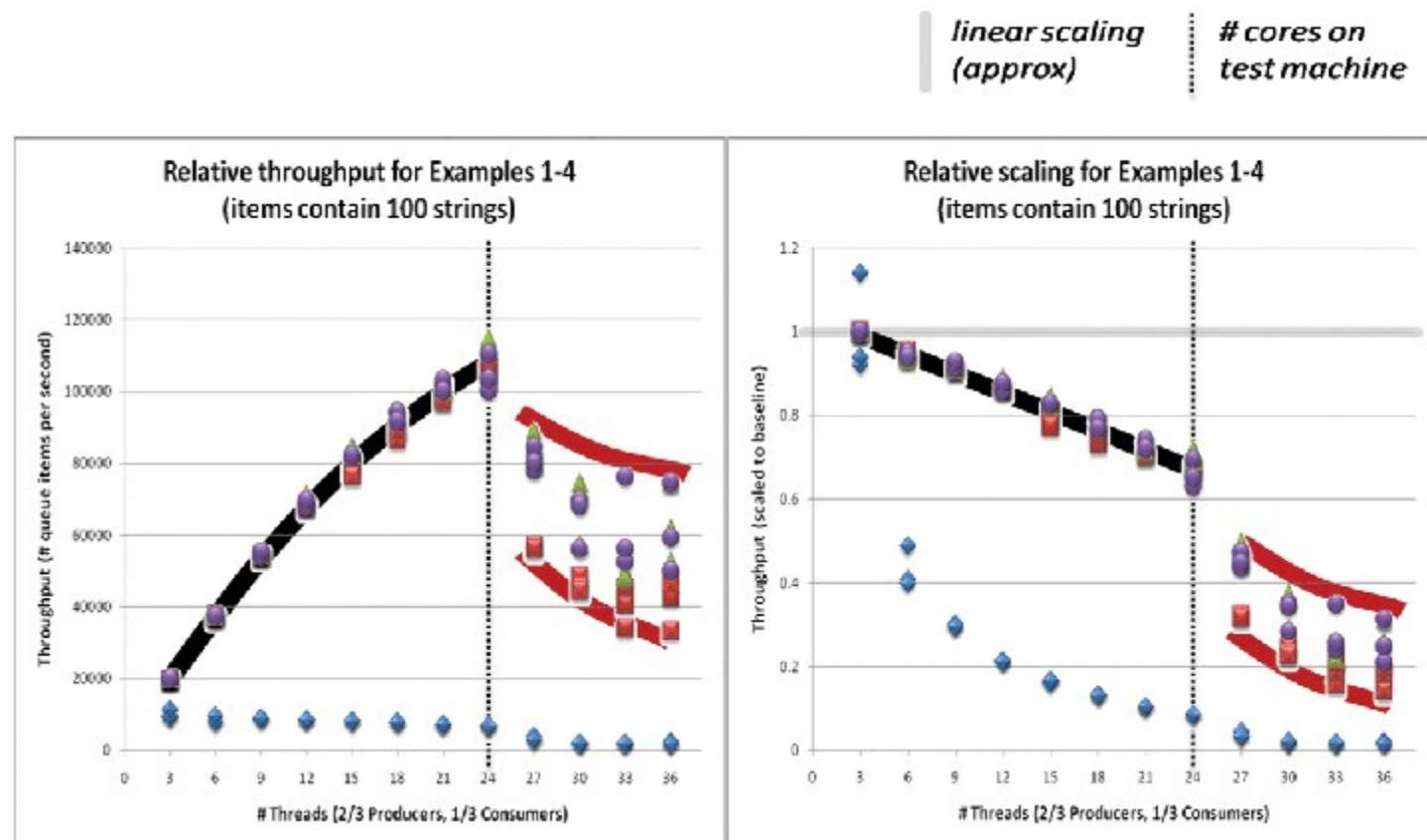


Figure 7: Another view of Figure 6, without misleading automatic trendlines.

Now the discontinuity is glaring and clear: On the left-hand graph, we move from a region of consistent and tight throughput increase through a wall, beyond which we experience a sudden drop and find ourselves in a new region where throughput is both dramatically lower and far less consistent and predictable—for example, multiple runs of the same Example 4 code at >24 threads shows dramatically variable results. On the right-hand graph, we move from a region of good and linearly decreasing scalability through a sudden drop, beyond which lies a new region where scalability too is both much lower and more variable.

What Have We Learned?

To improve scalability, we need to minimize contention:

- Reduce the size of critical sections so that we can get less contention and more concurrency among client threads.
- Reduce sharing of the same data by isolating threads to use different parts of a data structure. In our case, we moved the responsibility for cleanup from the producer to the consumer so that consumers touch only the head and producers touch only the tail.
- Reduce false sharing of different data on the same cache line, by adding padding to ensure that two separate variables that should be able to be used concurrently by different threads are also far enough apart in memory.

To understand our code's scalability, we need to know what to measure and what to look for in the results:

- Identify the key different kinds of work (here, producer threads and consumer threads), and then use stress tests to measure the impact of having different quantities and combinations of these in our workload.
- Identify the different kinds of data (here, representative "small" and "large" queue items), and then vary those to measure their impact.
- Measure total throughput, or items handled per unit time.
- Look for scalability, or the change in throughput as we add more threads. Does using more threads do more total work? Why or why not? In what directions, and for what combinations of workloads?
- Look for contention, or the interference between multiple threads trying to do work concurrently.
- Watch for the cost of oversubscription, and eliminate it either algorithmically or by limiting the actual amount of concurrency to avoid it altogether.
- Beware of overreliance on automated trendlines. Apply them only after first examining the raw data.

Be a scientist: Gather data. Analyze it. Especially when it comes to parallelism and scalability, there's just no substitute for the advice to measure, measure, measure, and understand what the results mean. Putting together test harnesses and generating and analyzing numbers is work, but the work will reward you with a priceless understanding of how your code actually runs, especially on parallel hardware—an understanding you will never gain from just reading the code or in any other way. And then, at the end, you will ship high-quality parallel code not because you think it's fast enough, but because you know under what circumstances it is and isn't (there will always be an "isn't"), and why.

Notes

[1] H. Sutter. "Writing Lock-Free Code: A Corrected Queue" (*DDJ*, October 2008).

[2] H. Sutter. "Maximize Locality, Minimize Contention" (*DDJ*, September 2008). www.ddj.com/architect/208200273.